

JADE 최소 필수 구현 기반 프로젝트 진행 계획

Codex 구현 지시용 Markdown 및 팀 공유용 문서

KCG 지로공 텀프 프로젝트

2026-05-29

차 례

0. 이 문서의 목적	2
1. 최종 방향 요약	2
1.1 최종 프로젝트 정의	2
1.2 지금 바로 구현해야 할 최소 필수 조합	3
2. 이전 계획에서 수정된 부분	3
2.1 왜 수정했는가	3
2.2 변경 전/후 비교	4
3. 최종 MVP 범위 고정	4
3.1 이번 단계에서 반드시 들어갈 것	4
3.2 이번 단계에서 제외하고 hook 만 남길 것	5
4. 최종 시스템 흐름	5
5. 권장 프로젝트 코드 구조	6
6. 핵심 데이터 구조	7
6.1 DrawingPlan / Unit Action	7
6.2 TrajectoryPoint	8
6.3 JacobianMetrics	8
6.4 FeasibilityReport	8
6.5 MotionPolicy	9
6.6 ExecutionLog row	9
7. 설정 파일 기준	9
7.1 configs/frames.yaml	9
7.2 configs/jade.yaml	10
7.3 configs/isaac_scene.yaml	10
8. 구현 순서: 현재 상태에서 무엇을 어떻게 해야 하는가	11
Step 0. 브랜치와 실행 환경 정리	11
Step 1. DrawingPlan parser 정리	11
Step 2. Stroke sampler 와 time scaling 구현	11
Step 3. Frame transform 과 pen-tip offset 보정	12
Step 4. Jacobian metrics 구현	12
Step 5. Stroke Validator 와 Adaptive Policy 구현	13
Step 6. Offline JADE runner 완성	13
Step 7. Minimal IsaacFrankaBackend 구현	14
Step 8. Adaptive DLS executor 구현	14
Step 9. Planned-vs-Actual Evaluation 구현	15

9. 첫 Isaac Sim 시뮬레이션 시작 전략	15
9.1 첫 목표는 contact drawing 이 아니라 hover tracing	15
9.2 첫 시뮬레이션 실행 순서	16
9.3 Scene 담당자가 반드시 넘겨줘야 할 값	16
10. Codex 구현 지시 요약	16
11. 테스트 계획	17
11.1 Unit tests	17
11.2 Integration tests	17
11.3 Isaac smoke test	17
12. MVP 성공 기준	18
13. 나중에 추가할 확장 기능	18
13.1 Null-space posture control	18
13.2 Guarded pen-down	18
13.3 Hybrid position-force control	19
13.4 ROS2 연동	19
14. 팀 역할 분담 권장	19
15. 최종 보고서/발표에서 강조할 문장	19
16. 최종 체크리스트	20
코드 구현 전 체크리스트	20
Offline 완료 체크리스트	20
Isaac MVP 완료 체크리스트	20
17. 가장 중요한 결론	20

0. 이 문서의 목적

이 문서는 현재 프로젝트를 바로 Isaac Sim 시뮬레이션 단계로 가져가기 위한 최종 정리본이다. 핵심은 **최소 필수 구현을 먼저 완성하고**, force control, null-space posture, ROS2 연동 같은 고급 기능은 나중에 확장 가능한 구조로 남기는 것이다.

이 문서는 두 가지 용도로 작성되었다.

1. **Codex 구현 지시용**: 어떤 모듈과 파일을 만들고, 어떤 순서로 구현하고, 어떤 테스트를 통과해야 하는지 명확히 전달한다.
2. **팀 공유용**: 교수님 피드백을 반영해 어떤 방향으로 프로젝트를 수정했는지, 팀원이 각자 무엇을 해야 하는지 한눈에 이해할 수 있게 한다.

전제는 다음과 같다.

- 현재 프로젝트에는 LLM 이 drawing Unit Action 또는 DrawingPlan JSON 을 만드는 구조가 있다.
- 현재 프로젝트에는 trajectory, frame transform, differential IK, evaluation 관련 기본 모듈 또는 초안이 있다.
- Isaac Sim backend 는 아직 핵심 빈칸이며, 최종 실행을 위해 연결해야 한다.
- 프로젝트 폴더를 다시 분석하지 않고, 이전에 확인한 구조와 교수님 피드백 자료 2 를 기준으로 방향을 확정한다.

1. 최종 방향 요약

1.1 최종 프로젝트 정의

기존의 약한 설명은 다음과 같다.

LLM 이 그림 명령을 Unit Action 으로 바꾸고, IK 라이브러리를 이용해 Franka 가 그림을 그린다.

이 설명만으로는 프로젝트 contribution 이 단순 API call 처럼 보일 수 있다. 최종 방향은 다음과 같이 수정한다.

LLM-generated Drawing Unit Action 을 JADE 라는 Jacobian-aware execution layer 에 통과시켜 stroke trajectory 를 만들고, Jacobian/manipulability/singularity/joint-limit 관점에서 검증한 뒤, 위험 구간에서는 damping 과 speed 를 조정하고, Isaac Sim 실행 후 planned-vs-actual trajectory 를 정량 평가한다. 프로젝트의 핵심 이름은 다음으로 고정한다.

JADE: Jacobian-Aware Drawing Executor

최종 contribution 문장:

본 프로젝트는 LLM 이 만든 drawing Unit Action 을 IK 라이브러리에 바로 넘기지 않는다. 각 stroke 를 JADE 라는 Jacobian-aware execution layer 에 통과시켜 manipulability, singularity risk, joint-limit margin 을 계산하고, 위험 구간에서는 속도와 damping 을 조정한다. 실행 후에는 Isaac Sim 에서 기록한 실제 pen-tip trajectory 와 계획 경로를 비교하여 정량 평가한다.

1.2 지금 바로 구현해야 할 최소 필수 조합

자료 2 의 결론을 반영하면, 최소 필수 구현은 다음 세 가지다.

구분	필수 구현	목적
1	Jacobian-aware Stroke Validator	각 stroke 가 실행 가능한지, singularity 와 joint limit 위험이 있는지 사전 평가
2	Craig-style Drawing Trajectory Generator	line/arc Unit Action 을 smooth time-scaled trajectory 로 변환
3	Planned-vs-Actual Trajectory Evaluation	Isaac Sim 에서 실제 실행 결과를 그래프와 수치로 평가

여기에 바로 Isaac Sim 실행을 시작하기 위해 다음 두 가지를 추가로 포함한다.

구분	필수 구현	목적
4	Minimal Adaptive DLS Executor	DLS 기반으로 desired pen-tip trajectory 를 joint command 로 변환
5	Minimal IsaacFrankaBackend	Isaac Sim 에서 q, qdot, EE pose, Jacobian 을 읽고 command 를 보냄

즉 MVP 는 아래 5 개다.

MVP = Stroke trajectory generator
+ Jacobian-aware validator
+ Adaptive DLS executor
+ Minimal Isaac backend
+ Planned-vs-actual evaluator

2. 이전 계획에서 수정된 부분

2.1 왜 수정했는가

교수님 피드백의 핵심은“라이브러리를 쓰지 말라”가 아니라, 라이브러리 호출만이 프로젝트의 전부가 되면 안 된다는 것이다. 따라서 Isaac Sim, IK utility, ROS2, Franka library 는 backend 로 사용하되, 우리가 직접 만든 robotics layer 가 있어야 한다.

수정 후 프로젝트는 다음 질문에 답해야 한다.

이 Unit Action 을 로봇이 안전하고 안정적으로 실행할 수 있는가?
 실행 전에 어떤 위험을 계산했는가?
 위험 구간에서 어떤 보정을 했는가?
 실행 후 계획 경로와 실제 경로가 얼마나 차이 났는가?

2.2 변경 전/후 비교

항목	이전 방향	수정된 최종 방향
프로젝트 중심	LLM + Franka drawing simulation	JADE: Jacobian-aware validation/correction/evaluation layer
Isaac Sim 의 역할 IK library 의 역할	최종 구현의 중심 주요 실행 엔진	JADE 를 검증하는 실행 환경 nominal backend. 결과를 JADE 가 검증하고 보정
force control ROS2/franka_ros2	초반부터 중요 기능 설치와 연동을 중요하게 고려	나중에 추가할 advanced feature main path 가 아님. 필요하면 optional extension
최우선 결과	Franka 가 움직이는 영상	planned vs actual path, error, manipulability, condition number 그래프
발표 메시지	“LLM 이 로봇을 움직인다”	“LLM action 을 JADE 가 로봇공학적으로 실행 가능하게 만든다”

3. 최종 MVP 범위 고정

3.1 이번 단계에서 반드시 들어갈 것

이번 단계에서 반드시 구현할 기능은 다음과 같다.

1. Unit Action parser/validator

- LLM planner 가 만든 DrawingPlan JSON 을 읽는다.
- 지원 action 은 move_to_start, pen_down, draw_line, draw_arc, pen_up 으로 제한한다.
- 알 수 없는 action, 누락된 geometry, 단위 오류를 실행 전에 잡는다.

2. Stroke sampler

- draw_line: 시작점과 끝점을 board frame waypoint 로 샘플링한다.
- draw_arc: 중심, 반지름, 시작각, 끝각으로 waypoint 를 샘플링한다.
- 모든 geometry 는 meter 단위로 통일한다.

3. Time scaling

- uniform waypoint 만 쓰지 않고 cubic 또는 quintic time scaling 을 적용한다.
- pen-down drawing stroke 는 일정하고 느리게, pen-up move 는 상대적으로 빠르게 움직인다.
- sharp corner 근처에서는 speed 를 줄이는 hook 을 남긴다.

4. Frame transform / tool offset compensation

- T_base_board 로 board frame path 를 robot base frame path 로 변환한다.
- T_ee_tip 으로 pen tip target 을 end-effector target 으로 변환한다.
- 모든 transform 은 4x4 homogeneous matrix 로 통일한다.

5. Jacobian metrics

- 각 waypoint 또는 일정 간격 waypoint 에서 Jacobian 을 얻는다.
- singular values, minimum singular value, condition number, manipulability, joint-limit margin 을 계산한다.

6. Stroke feasibility report

- 각 stroke 에 대해 OK, WARNING, FAIL 을 반환한다.
- 위험 이유와 추천 speed scale, DLS damping 값을 기록한다.

7. Adaptive DLS executor

- Damped least-squares 로 desired Cartesian motion 을 joint command 로 변환한다.
- 위험 구간에서는 damping 을 키우고 speed 를 줄인다.
- joint velocity 와 position step 을 clamp 한다.

8. Minimal Isaac backend

- Isaac Sim 에서 Franka state, end-effector pose, Jacobian 을 읽는다.
- joint position target 또는 joint velocity target 을 보낸다.
- actual pen-tip pose 를 logging 한다.

9. Execution logger and evaluator

- desired path, actual path, error, q, qdot, singular value, condition number, damping, speed scale 을 CSV 로 저장한다.
- planned vs actual XY path, error over time, manipulability/condition over time 그래프를 만든다.

10. 첫 시뮬레이션 demo

- 처음에는 contact drawing 이 아니라 **hover tracing** 으로 시작한다.
- 펜이 board 위 1-2 cm 를 따라가며 원 또는 사각형 trajectory 를 추종한다.
- 그 다음 pen_down/contact/force 를 추가한다.

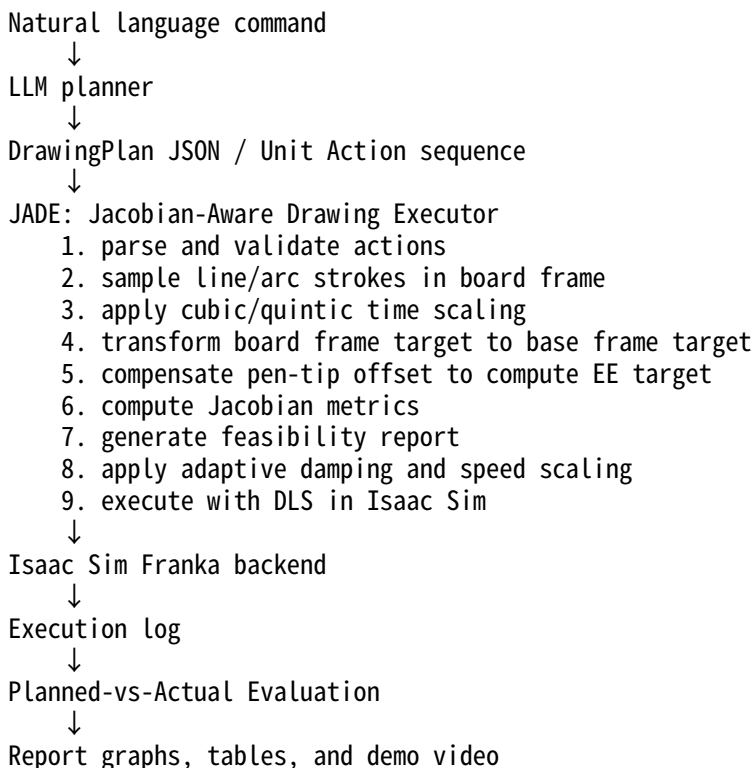
3.2 이번 단계에서 제외하고 hook 만 남길 것

다음 기능은 구조상 hook 은 만들되, MVP 완료 전에는 필수로 구현하지 않는다.

기능	이번 단계 처리	나중에 추가할 내용
Force control	optional field 와 interface 만 남김	guarded pen-down, normal force tracking
Null-space posture control	placeholder 함수만 남김	joint-limit avoidance, manipulability maximization
ROS2/franka_ros2	main path 에서 제외	필요 시 Isaac ROS bridge 로 연동
Full hybrid position-force control	제외	$\tau = J^T F + g(q)$ 기반 extension
Complex handwriting	제외	circle, square, letter A 이후 추가
Real robot execution	제외	Isaac 검증 후 future work

4. 최종 시스템 흐름

최종 pipeline 은 다음으로 고정한다.



핵심 원칙:

LLM output 은 trajectory 가 아니다.

LLM output 은 symbolic Unit Action 이다.

Trajectory, validation, correction, execution, evaluation 은 JADE 가 담당한다.

5. 권장 프로젝트 코드 구조

Codex 는 현재 폴더 구조를 크게 부수지 말고, 아래 구조를 목표로 정리한다. 기존 파일명이 다르면 기존 구조를 최대한 유지 하되 동일한 역할의 모듈로 매핑한다.

```
project_root/
├── franka_llm_drawing/
│   ├── llm_bridge/
│   │   ├── __init__.py
│   │   └── drawing_plan_parser.py
│   ├── trajectory/
│   │   ├── __init__.py
│   │   ├── stroke_sampler.py
│   │   ├── time_scaling.py
│   │   └── trajectory_types.py
│   ├── frames/
│   │   ├── __init__.py
│   │   └── transforms.py
│   ├── jade/
│   │   ├── __init__.py
│   │   ├── jacobian_metrics.py
│   │   ├── stroke_validator.py
│   │   ├── adaptive_policy.py
│   │   ├── adaptive_dls_executor.py
│   │   └── reports.py
│   ├── sim/
│   │   ├── __init__.py
│   │   ├── robot_backend.py
│   │   ├── mock_backend.py
│   │   └── isaac_backend.py
│   └── evaluation/
│       ├── __init__.py
│       ├── execution_logger.py
│       ├── metrics.py
│       └── plots.py
├── configs/
│   ├── frames.yaml
│   ├── jade.yaml
│   ├── controller_gains.yaml
│   └── isaac_scene.yaml
├── examples/
│   ├── sample_plans/
│   │   ├── circle.json
│   │   ├── square.json
│   │   └── letter_a.json
│   ├── run_jade_offline.py
│   └── run_jade_isaac.py
└── tests/
    ├── test_stroke_sampler.py
    ├── test_time_scaling.py
    └── test_jacobian_metrics.py
```

```

├── test_stroke_validator.py
├── test_evaluation_metrics.py
├── outputs/
└── .gitkeep

```

중요한 점:

- llm_bridge 는 LLM planner 를 대체하지 않는다. 기존 LLM output 을 robot-control core 로 가져오는 역할만 한다.
- trajectory 는 geometric stroke 를 time-sampled desired path 로 바꾼다.
- jade 가 이번 프로젝트의 핵심 contribution 이다.
- sim 은 Isaac 또는 mock backend 를 감싸는 adapter 이다.
- evaluation 은 발표용 그래프와 표를 만든다.

6. 핵심 데이터 구조

6.1 DrawingPlan / Unit Action

LLM 이 생성하는 JSON 은 너무 자유로우면 안 된다. Codex 는 다음 schema 를 기준으로 parser 와 validator 를 만든다.

```

{
  "plan_id": "circle_demo_001",
  "units": "m",
  "board_frame": "board",
  "actions": [
    {
      "id": "a0",
      "type": "move_to_start",
      "target": [0.05, 0.00, 0.02],
      "speed": 0.10
    },
    {
      "id": "a1",
      "type": "pen_down",
      "target_z": 0.00,
      "speed": 0.02
    },
    {
      "id": "a2",
      "type": "draw_arc",
      "center": [0.00, 0.00, 0.00],
      "radius": 0.05,
      "theta_start_deg": 0.0,
      "theta_end_deg": 360.0,
      "clockwise": false,
      "speed": 0.03
    },
    {
      "id": "a3",
      "type": "pen_up",
      "lift_distance": 0.02,
      "speed": 0.08
    }
  ]
}

```

MVP 에서 반드시 지원할 action:

```

move_to_start
draw_line

```

draw_arc
pen_down
pen_up

MVP 에서 무시하거나 error 처리할 action:

fill_shape
write_text
change_color
freeform_curve

6.2 TrajectoryPoint

```
@dataclass
class TrajectoryPoint:
    t: float
    stroke_id: str
    action_type: str
    p_board: np.ndarray          # shape (3,)
    R_board_tip: np.ndarray      # shape (3, 3)
    p_base: np.ndarray           # shape (3,)
    R_base_tip: np.ndarray       # shape (3, 3)
    T_base_tip: np.ndarray       # shape (4, 4)
    T_base_ee: np.ndarray        # shape (4, 4)
    speed_scale: float = 1.0
    lambda_dls: float | None = None
```

6.3 JacobianMetrics

```
@dataclass
class JacobianMetrics:
    stroke_id: str
    waypoint_index: int
    sigma_min: float
    sigma_max: float
    condition_number: float
    manipulability: float
    joint_limit_margin: float
    q_norm: float
    qdot_norm_est: float | None = None
```

MVP 에서는 Jacobian 이 full 6x7 이면 full Jacobian 으로 계산한다. Isaac API 또는 mock backend 에서 translational Jacobian 만 안정적으로 얻을 수 있으면, 우선 3x7 J_pos 를 사용하고 문서에 명시한다.

6.4 FeasibilityReport

```
@dataclass
class FeasibilityReport:
    stroke_id: str
    status: Literal["OK", "WARNING", "FAIL"]
    min_manipulability: float
    max_condition_number: float
    min_sigma: float
    min_joint_limit_margin: float
    recommended_speed_scale: float
    recommended_damping: float
    warning_waypoints: List[int]
```



```
fail_waypoints: list[int]
reasons: list[str]
```

6.5 MotionPolicy

```
@dataclass
class MotionPolicy:
    speed_scale: float
    lambda_dls: float
    max_qdot: float
    max_delta_q: float
    status: Literal["OK", "WARNING", "FAIL"]
    reason: str
```

6.6 ExecutionLog row

CSV log 는 최소 다음 column 을 포함한다.

```
t, stroke_id, action_type,
x_des, y_des, z_des,
x_act, y_act, z_act,
error_x, error_y, error_z, error_norm,
q1, q2, q3, q4, q5, q6, q7,
qdot1, qdot2, qdot3, qdot4, qdot5, qdot6, qdot7,
sigma_min, condition_number, manipulability,
lambda_dls, speed_scale,
status
```

force sensor 가 준비되면 다음 column 을 추가한다.

```
normal_force, contact_flag, force_error
```

7. 설정 파일 기준

7.1 configs/frames.yaml

```
frames:
  base_frame: panda_link0
  ee_frame: panda_hand
  tip_frame: pen_tip
  board_frame: board

T_base_board:
  translation_m: [0.55, 0.00, 0.20]
  rpy_deg: [0.0, 0.0, 0.0]

T_ee_tip:
  translation_m: [0.00, 0.00, 0.12]
  rpy_deg: [0.0, 0.0, 0.0]

board:
  size_m: [0.30, 0.30]
  drawing_z_m: 0.0
  hover_z_m: 0.02
  normal_axis_base: [0.0, 0.0, 1.0]
```

주의:

- 위 값은 예시다. 환경/USD 담당자가 실제 Isaac scene 값으로 확정해야 한다.
- T_{ee_tip} 은 end-effector 에서 pen-tip 까지의 transform 이다.
- T_{base_board} 가 틀리면 모든 drawing path 가 잘못된 위치로 간다.

7.2 configs/jade.yaml

```
jade:
  sampling:
    dt: 0.02
    max_segment_length_m: 0.005
    time_scaling: quintic

  thresholds:
    sigma_min_warning: 0.03
    sigma_min_fail: 0.01
    condition_warning: 100.0
    condition_fail: 300.0
    manipulability_warning: 0.02
    joint_margin_warning_rad: 0.15
    joint_margin_fail_rad: 0.05

  adaptive_policy:
    lambda_min: 0.02
    lambda_max: 0.20
    speed_scale_min: 0.20
    speed_scale_warning: 0.50

  executor:
    max_qdot_rad_s: 1.0
    max_delta_q_rad: 0.03
    position_gain: 1.0
    orientation_gain: 0.3
```

처음에는 threshold 가 완벽하지 않아도 된다. 중요한 것은 값을 config 로 두어 실험 후 조정할 수 있게 만드는 것이다.

7.3 configs/isaac_scene.yaml

```
isaac:
  usd_path: "scenes/franka_drawing_scene.usd"
  robot_prim_path: "/World/Franka"
  board_prim_path: "/World/Board"
  pen_tip_prim_path: "/World/Franka/panda_hand/PenTip"
  camera_prim_path: "/World/Camera"
  physics_dt: 0.005
  control_dt: 0.02
  use_gpu: false

backend:
  command_mode: joint_position
  ee_frame_name: panda_hand
  joint_names:
    - panda_joint1
    - panda_joint2
    - panda_joint3
    - panda_joint4
    - panda_joint5
    - panda_joint6
```

- panda_joint7

8. 구현 순서: 현재 상태에서 무엇을 어떻게 해야 하는가

Step 0. 브랜치와 실행 환경 정리

목표:

기존 코드를 망가뜨리지 않고 JADE MVP 구현을 시작할 수 있게 한다.

해야 할 일:

```
git checkout -b feature/jade-mvp
python -m pip install -e ".[dev]"
pytest -q
```

만약 editable install 이 아직 정리되어 있지 않다면, Codex 는 pyproject.toml 또는 setup.cfg 를 정리해 다음 명령이 동작하게 만든다.

```
python -m pip install -e .
```

완료 기준:

- 기존 테스트가 깨지지 않는다.
- `python -c "import franka_llm_drawing"` 가 성공한다.

Step 1. DrawingPlan parser 정리

목표:

LLM planner output 을 JADE 가 안정적으로 읽을 수 있게 한다.

해야 할 일:

- `franka_llm_drawing/llm_bridge/drawing_plan_parser.py` 에 dataclass 와 parser 를 만든다.
- JSON 에 누락된 필드가 있으면 명확한 error message 를 낸다.
- action type 별 required field 를 검사한다.
- 단위는 meter 로 통일한다.

완료 기준:

```
python examples/run_jade_offline.py --plan examples/sample_plans/circle.json --dry-run
```

이 명령이 action list 와 validation summary 를 출력해야 한다.

Step 2. Stroke sampler 와 time scaling 구현

목표:

`draw_line/draw_arc` 를 smooth desired trajectory 로 변환한다.

해야 할 일:

- `stroke_sampler.py`
 - `sample_line(start, end, max_segment_length)`
 - `sample_arc(center, radius, theta0, theta1, max_segment_length)`
- `time_scaling.py`
 - $\text{cubic}_s(t, T) = 3 \tau^2 - 2 \tau^3$
 - $\text{quintic}_s(t, T) = 10 \tau^3 - 15 \tau^4 + 6 \tau^5$
 - `assign_timestamps(points, speed, dt, method)`

수식:

line: $p(s) = p_0 + s(p_1 - p_0)$
 arc: $p(s) = c + r [\cos(\theta(s)), \sin(\theta(s)), z]$
 quintic: $s(t) = 10 \tau^3 - 15 \tau^4 + 6 \tau^5$

완료 기준:

- line endpoint 가 정확히 start/end 와 일치한다.
- arc endpoint 가 $\theta_{start}/\theta_{end}$ 와 일치한다.
- trajectory timestamp 가 strictly increasing 이다.
- velocity 가 시작/끝에서 급격히 튀지 않는다.

Step 3. Frame transform 과 pen-tip offset 보정

목표:

board-frame pen-tip target 을 robot-base-frame EE target 으로 변환한다.

해야 할 일:

- frames/transforms.py 에 다음 함수를 둔다.

```
def make_transform(translation: np.ndarray, rotation: np.ndarray) -> np.ndarray: ...
def invert_transform(T: np.ndarray) -> np.ndarray: ...
def transform_point(T: np.ndarray, p: np.ndarray) -> np.ndarray: ...
def compute_T_base_tip(T_base_board, T_board_tip) -> np.ndarray: ...
def compute_T_base_ee(T_base_tip, T_ee_tip) -> np.ndarray: ...
```

핵심 식:

```
T_base_tip = T_base_board @ T_board_tip
T_base_ee = T_base_tip @ inverse(T_ee_tip)
```

완료 기준:

- identity transform test 통과.
- inverse transform test 통과.
- known translation/rotation case 통과.
- pen-tip offset 이 적용되어 EE target 이 tip target 과 다르게 계산된다.

Step 4. Jacobian metrics 구현

목표:

JADE 가 각 waypoint 의 kinematic risk 를 계산할 수 있게 한다.

해야 할 일:

- jade/jacobian_metrics.py 구현.
- 입력: J, q, joint_limits.
- 출력: singular values, condition number, manipulability, joint limit margin.

기본 계산:

```
singular_values = svd(J)
sigma_min = min(singular_values)
sigma_max = max(singular_values)
condition_number = sigma_max / max(sigma_min, eps)
manipulability = sqrt(det(J_pos @ J_pos.T))
joint_limit_margin = min(q - q_lower, q_upper - q)
```

주의:

- J 가 6x7 이면 position part $J_{pos} = J[0:3, :]$ 를 manipulability 계산에 우선 사용한다.
- $\det(J_{pos} @ J_{pos}.T)$ 가 numerical error 로 음수가 되면 $\max(\text{value}, 0.0)$ 처리한다.
- condition number 가 무한대가 되지 않도록 eps 를 둔다.

완료 기준:

- well-conditioned mock Jacobian 에서 status OK.
- rank-deficient Jacobian 에서 condition number 가 크게 나온다.
- joint limit 근처 q 에서 margin warning 이 나온다.

Step 5. Stroke Validator 와 Adaptive Policy 구현

목표:

각 stroke 실행 전에 OK/WARNING/FAIL 과 recommended speed/damping 을 계산한다.

해야 할 일:

- jade/stroke_validator.py
 - validate_stroke(trjectory_points, backend_or_model, q_seed, config)
- jade/adaptive_policy.py
 - recommend_policy(metrics, thresholds)

정책 예시:

```
if sigma_min < sigma_min_fail or condition > condition_fail or joint_margin < joint_margin_fail:
    status = FAIL
elif sigma_min < sigma_min_warning or condition > condition_warning or joint_margin < joint_margin_warning:
    status = WARNING
else:
    status = OK
```

Adaptive damping:

```
lambda = lambda_min + alpha / (manipulability + eps)
lambda = clip(lambda, lambda_min, lambda_max)
```

Speed scaling:

```
OK:      speed_scale = 1.0
WARNING: speed_scale = 0.5 or continuous scale
FAIL:    speed_scale = 0.0 and do not execute unless override is enabled
```

완료 기준:

- run_jade_offline.py 가 stroke 별 FeasibilityReport 를 출력한다.
- report JSON 또는 CSV 가 outputs/<run_id>/feasibility_report.json 에 저장된다.

Step 6. Offline JADE runner 완성

목표:

Isaac 없이도 JADE contribution 을 먼저 검증한다.

해야 할 일:

- examples/run_jade_offline.py 구현.
- 입력: plan JSON, config files.
- 출력:
 - sampled trajectory CSV
 - feasibility report JSON
 - planned path plot
 - manipulability/condition plot, mock backend 이면 mock 값으로 표시

실행 예시:

```
python examples/run_jade_offline.py \
--plan examples/sample_plans/circle.json \
--frames configs/frames.yaml \
```

```
--jade configs/jade.yaml \
--out outputs/offline_circle
```

완료 기준:

- Isaac 없이도 trajectory 와 report 가 생성된다.
- 교수님에게 “우리가 만든 validation/evaluation layer”를 코드와 그래프로 설명할 수 있다.

Step 7. Minimal IsaacFrankaBackend 구현

목표:

JADE trajectory 를 Isaac Sim Franka 에 연결한다.

MVP backend interface:

```
class RobotBackend(Protocol):
    def get_q(self) -> np.ndarray: ...
    def get_qdot(self) -> np.ndarray: ...
    def get_ee_pose(self) -> np.ndarray: ... # 4x4 T_base_ee
    def get_jacobian(self) -> np.ndarray: ... # 6x7 or 3x7
    def send_joint_position_target(self, q: np.ndarray) -> None: ...
    def send_joint_velocity_target(self, qdot: np.ndarray) -> None: ...
    def step(self) -> None: ...
```

Isaac backend 는 처음부터 완벽할 필요가 없다. 우선 다음만 되면 된다.

- Franka articulation handle 가져오기.
- 현재 joint position/velocity 읽기.
- 현재 EE pose 읽기.
- geometric Jacobian 읽기.
- joint position target 보내기.
- simulation step 진행.

주의:

- Isaac API 버전에 따라 Jacobian 반환 frame 과 joint order 가 다를 수 있다.
- backend docstring 에 Jacobian is expressed in base frame 또는 실제 기준 frame 을 반드시 적는다.
- q vector 의 joint order 가 panda_joint1 부터 panda_joint7 인지 확인한다.

완료 기준:

```
python examples/run_jade_isaac.py --plan examples/sample_plans/circle.json --mode hover --out outputs/isaac_circle
```

이 명령이 Isaac 에서 Franka 를 움직이고 CSV log 를 생성해야 한다.

Step 8. Adaptive DLS executor 구현

목표:

JADE corrected trajectory 를 실제 joint command 로 변환한다.

DLS 기본식:

$$\dot{q} = J.T @ \text{inv}(J @ J.T + \lambda^2 I) @ \dot{x}_{\text{des}}$$

MVP 에서는 position tracking 을 먼저 한다.

```
x_error = p_des - p_current
xdot_cmd = Kp * x_error
qdot_cmd = DLS(J_pos, xdot_cmd, lambda)
qdot_cmd = clamp(qdot_cmd, max_qdot)
q_target = q_current + qdot_cmd * dt
q_target = clamp(q_target, joint_limits)
```

orientation 은 다음 중 하나로 처리한다.

- 첫 시뮬레이션: Franka EE orientation 을 fixed drawing orientation 으로 유지.
- 가능하면: 6D pose error 를 사용해 position + orientation DLS 실행.

처음부터 full orientation control 이 안 되면, hover tracing 성공을 먼저 목표로 한다.

완료 기준:

- 원 또는 사각형 hover tracing 에서 pen-tip actual path 가 planned path 와 유사하게 나온다.
- $\dot{q}$ norm 이 config limit 을 넘지 않는다.
- λ_{dls} 와 speed_scale 이 log 에 기록된다.

Step 9. Planned-vs-Actual Evaluation 구현

목표:

실행 결과를 발표 가능한 그래프와 표로 만든다.

해야 할 일:

- evaluation/metrics.py
 - mean tracking error
 - max tracking error
 - RMSE
 - completion rate
 - max $\dot{q}$ norm
 - min sigma
 - max condition number
- evaluation/plots.py
 - planned vs actual XY path
 - tracking error over time
 - $\dot{q}$ norm over time
 - sigma_min or condition number over time
 - lambda and speed_scale over time

완료 기준:

outputs/isaac_circle/에 다음 파일이 있어야 한다.

execution_log.csv
summary_metrics.json
planned_vs_actual_xy.png
tracking_error.png
jacobian_condition.png
adaptive_policy.png

9. 첫 Isaac Sim 시뮬레이션 시작 전략

9.1 첫 목표는 contact drawing 이 아니라 hover tracing

바로 pen contact 와 force sensor 를 붙이면 디버깅 지점이 너무 많다. 첫 Isaac demo 는 다음으로 고정한다.

Mode: hover tracing
Shape: circle, radius 5 cm
Board: visual plane only, collision 은 있어도 무관
Pen tip: board 위 1-2 cm 유지
Control: DLS + joint position target
Evaluation: desired vs actual XY path, error over time

이렇게 하면 다음을 분리해서 확인할 수 있다.

- LLM plan 이 잘 들어오는가?
- trajectory sampling 이 맞는가?

- frame transform 이 맞는가?
- pen-tip offset 이 맞는가?
- Jacobian 과 q order 가 맞는가?
- DLS command 가 Franka 를 움직이는가?
- logging 과 plotting 이 되는가?

hover tracing 이 성공한 다음에 pen_down 과 contact 를 추가한다.

9.2 첫 시뮬레이션 실행 순서

1. Isaac scene 을 연다.
2. Franka 가 home pose 에 있는지 확인한다.
3. board frame 과 base frame 위치를 config 에 맞춘다.
4. pen tip marker 를 EE 에 붙인다.
5. circle plan 을 load 한다.
6. JADE 가 trajectory 와 feasibility report 를 만든다.
7. FAIL 이 없으면 DLS executor 가 hover trajectory 를 실행한다.
8. execution log 를 저장한다.
9. planned-vs-actual plot 을 만든다.
10. 영상 녹화 또는 화면 캡처를 한다.

9.3 Scene 담당자가 반드시 넘겨줘야 할 값

환경/USD 담당자는 다음 값을 코드 담당자에게 넘겨야 한다.

```
robot_prim_path
board_prim_path
pen_tip_prim_path
camera_prim_path
joint_names
end_effector_frame_name
T_base_board
T_ee_tip
board_size_m
board_normal_axis
physics_dt
control_dt
```

이 중 가장 중요한 것은 다음 두 가지다.

```
T_base_board
T_ee_tip
```

이 두 값이 틀리면 trajectory 와 IK 가 맞아도 그림 위치가 틀어진다.

10. Codex 구현 지시 요약

Codex 에게는 아래 지시를 그대로 전달해도 된다.

Implement the JADE MVP without changing the high-level LLM planner behavior.
The goal is to make LLM-generated DrawingPlan JSON executable in Isaac Sim through an explicit robotics layer.

Required modules:

1. trajectory/stroke_sampler.py
2. trajectory/time_scaling.py
3. frames/transforms.py
4. jade/jacobian_metrics.py
5. jade/stroke_validator.py

6. jade/adaptive_policy.py
7. jade/adaptive_dls_executor.py
8. sim/robot_backend.py
9. sim/isaac_backend.py
10. evaluation/execution_logger.py
11. evaluation/metrics.py
12. evaluation/plots.py
13. examples/run_jade_offline.py
14. examples/run_jade_isaac.py

Do not make force control, ROS2 integration, or null-space optimization a blocker.
Add clean extension hooks, but first make hover tracing work in Isaac Sim.

Definition of done:

- pytest passes.
- Offline circle plan generates trajectory CSV and feasibility report.
- Isaac circle hover tracing runs with Franka.
- execution_log.csv is saved.
- planned-vs-actual XY plot and tracking error plot are generated.

11. 테스트 계획

11.1 Unit tests

필수 테스트:

테스트 파일	검증 내용
test_stroke_sampler.py	line/arc endpoint, sample count, meter 단위
test_time_scaling.py	cubic/quintic boundary condition, timestamp monotonicity
test_transforms.py	inverse transform, T_base_tip, T_base_ee 계산
test_jacobian_metrics.py	singular value, condition number, manipulability, joint margin
test_stroke_validator.py	OK/WARNING/FAIL threshold logic
test_evaluation_metrics.py	RMSE, max error, completion rate

11.2 Integration tests

```
python examples/run_jade_offline.py --plan examples/sample_plans/circle.json --out outputs/test_offline_circle
python examples/run_jade_offline.py --plan examples/sample_plans/square.json --out outputs/test_offline_square
```

성공 기준:

- output CSV/JSON/PNG 가 생성된다.
- report 에 stroke 별 status 가 나온다.
- path plot 이 shape 와 일치한다.

11.3 Isaac smoke test

```
python examples/run_jade_isaac.py \
  --plan examples/sample_plans/circle.json \
  --mode hover \
  --out outputs/smoke_isaac_circle
```

성공 기준:

- Franka 가 움직인다.

- execution log 가 저장된다.
- actual pen-tip path 가 기록된다.
- 그래프가 생성된다.

12. MVP 성공 기준

MVP 는 다음 조건을 만족하면 완료로 판단한다.

항목	성공 기준
Offline trajectory	circle/square/letter A 중 최소 2 개 shape 의 planned path 생성
Stroke validation	각 stroke 마다 feasibility report 출력
Adaptive policy	warning 구간에서 lambda 또는 speed_scale 변화가 log 에 기록
Isaac execution	circle hover tracing 이 Franka 로 실행
Evaluation	planned vs actual XY path 와 error graph 생성
Report 자료	summary metrics table 생성
Demo	30 초 이상 Isaac Sim 화면 녹화 가능

MVP 발표 시 핵심 그래프:

1. Planned vs actual XY path
2. Cartesian tracking error over time
3. Minimum singular value or condition number over time
4. DLS damping lambda and speed scale over time
5. Summary metrics table

13. 나중에 추가할 확장 기능

13.1 Null-space posture control

Franka 는 7-DOF 이므로 같은 pen-tip pose 에 대해 여러 joint solution 이 가능하다. MVP 이후 다음 식을 추가한다.

$$\dot{q} = J\dot{x} + (I - J\dot{J}) \dot{q}_{null}$$

$\dot{q}_{null}$ 목표:

- joint limit 에서 멀어지기
- manipulability 높이기
- wrist 와 board 충돌 위험 줄이기
- elbow 가 board 아래로 내려가지 않게 하기

13.2 Guarded pen-down

force sensor 또는 contact sensor 가 준비되면 pen_down 을 다음으로 바꾼다.

guarded_pen_down(force_threshold, approach_speed)

동작:

1. z 방향으로 천천히 접근
2. normal force 또는 contact flag 감지
3. threshold 이상이면 stop
4. 현재 z 를 drawing plane 으로 lock
5. 이후 x-y 는 position tracking, z 는 compliance/force-aware control

13.3 Hybrid position-force control

고급 기능으로 다음 구조를 추가한다.

x-y direction: position control
z normal direction: force or compliance control
joint torque: $\tau = J.T @ \text{wrench} + \text{gravity_compensation}$

단, 이 기능은 MVP 이후 추가한다. force control 이 늦어져도 프로젝트 contribution 은 JADE validator/evaluator 로 유지된다.

13.4 ROS2 연동

ROS2 Humble/Jazzy, franka_ros2, MoveIt 연동은 main path 가 아니다. 필요한 경우 다음 단계에서만 추가한다.

JADE output -> ROS2 trajectory message -> MoveIt / controller -> Isaac or real robot

14. 팀 역할 분담 권장

담당	해야 할 일	MVP 산출물
LLM planner 담당	DrawingPlan JSON schema 맞추기, sample plans 생성	circle/square/letter A JSON
JADE/core 담당	sampler, time scaling, validator, adaptive DLS 구현	offline JADE report, DLS executor
Isaac/USD 담당	Franka, board, pen tip marker, camera, prim path, frame 값 확정	scene file, frames.yaml, isaac_scene.yaml
Evaluation/report 담당	logger, plots, metrics, result table, demo 영상 정리	CSV, PNG graphs, summary table, video

팀 간 interface 는 다음 파일로 고정한다.

examples/sample_plans/*.json
configs/frames.yaml
configs/jade.yaml
configs/isaac_scene.yaml
outputs/<run_id>/execution_log.csv

15. 최종 보고서/발표에서 강조할 문장

한국어 발표 문장:

본 프로젝트는 LLM 이 만든 drawing Unit Action 을 IK 라이브러리에 그대로 넘기지 않습니다. 각 stroke 를 JADE 라는 Jacobian-aware execution layer 에 통과시켜 trajectory 를 생성하고, manipulability, singularity risk, joint-limit margin 을 계산합니다. 위험 구간에서는 drawing speed 와 DLS damping 을 조정하며, Isaac Sim 실행 후 실제 pen-tip trajectory 와 planned trajectory 를 비교해 정량적으로 평가합니다.

영어 발표 문장:

Our project does not simply pass LLM-generated drawing commands to an IK library. We introduce JADE, a Jacobian-Aware Drawing Executor, which samples each stroke, evaluates Jacobian quality, manipulability, singularity risk, and joint-limit margin, adapts speed and damping when needed, and quantitatively compares the executed pen-tip trajectory with the planned path in Isaac Sim.

16. 최종 체크리스트

코드 구현 전 체크리스트

- [] sample DrawingPlan JSON 3 개 확정: circle, square, letter A
- [] frames.yaml 의 T_base_board 예시값 입력
- [] T_ee_tip 예시값 입력
- [] jade.yaml threshold 와 gains 입력
- [] Isaac scene 의 robot_prim_path 확정
- [] pen_tip marker prim 확정

Offline 완료 체크리스트

- [] DrawingPlan parser 통과
- [] line/arc sampler 통과
- [] time scaling 통과
- [] frame transform 통과
- [] feasibility report 생성
- [] planned path plot 생성

Isaac MVP 완료 체크리스트

- [] Franka q/qdot 읽기 성공
- [] EE pose 읽기 성공
- [] Jacobian 읽기 성공
- [] joint target command 성공
- [] circle hover tracing 실행
- [] execution_log.csv 저장
- [] planned-vs-actual plot 생성
- [] tracking error plot 생성
- [] demo video 녹화

17. 가장 중요한 결론

지금부터는 다음 순서를 절대 바꾸지 않는 것이 좋다.

1. JADE offline MVP 완성
2. Isaac hover tracing 연결
3. planned-vs-actual evaluation 생성
4. 그 다음 force/contact/null-space/ROS2 확장

즉, 현재 프로젝트의 최종 방향은 다음으로 확정한다.

LLM planner

→ Drawing Unit Action

→ JADE minimum core

→ stroke sampling

→ smooth trajectory generation

→ Jacobian-aware validation

→ adaptive damping/speed scaling

→ Isaac Sim hover tracing

→ planned-vs-actual quantitative evaluation

→ optional contact/force extensions

이 방향으로 진행하면 교수님 피드백인 “단순 라이브러리 호출이 아닌 수업 기반 contribution”을 가장 안정적으로 반영하면 서도, 바로 Isaac Sim 시뮬레이션을 시작할 수 있다.